Teaching the OOM killer about control groups

Did you know...?

LWN.net is a subscriber-supported publication; we rely on subscribers to keep the entire operation going. Please help out by buying a subscription and keeping LWN on the net.

By **Jonathan Corbet**
July 27, 2018

The kernel's out-of-memory (OOM) killer is summoned when the system runs short of free memory and is unable to proceed without killing one or more processes. As might be expected, the policy decisions around *which* processes should be targeted have engendered controversy for as long as the OOM killer has existed. The 4.19 development cycle is likely to include a new OOM-killer implementation that targets control groups rather than individual processes, but it turns out that there is significant disagreement over how the OOM killer and control groups should interact.

To simplify a bit: when the OOM killer is invoked, it tries to pick the process whose demise will free the most memory while causing the least misery for users of the system. The heuristics used to make this selection have varied considerably over time — it was once remarked that each developer who changes the heuristics makes them work for their use case while ruining things for everybody else. In current kernels, the heuristics implemented in `oom_badness()` are relatively simple: sum up the amount of memory used by a process, then scale it by the process's `oom_score_adj` value. That value, found in the process's `/proc` directory, can be tweaked by system administrators to make specific processes more or less attractive as an OOM-killer target.

No OOM-killer implementation is perfect, and this one is no exception. One problem is that it does not pay attention to how much memory a particular *user* has allocated; it only looks at specific processes. If user A has a single large process while user B has 100 smaller ones, the OOM killer will invariably target A's process, even if B is using far more memory overall. That behavior is tolerable on a single-user system, but it is less than optimal on a large system running containers on behalf of multiple users.

Control-group awareness

To address this issue, Roman Gushchin has introduced the control-group-aware OOM killer. It modifies the OOM-kill algorithm in a fairly straightforward way: first, the control group with the largest memory consumption is found, then the largest process running within that group is killed. There is also a new knob added to control groups called `memory.oom_group`; if it is set to a non-zero value, the OOM killer will kill *all* processes running within the targeted group instead of just the largest one. This flag is useful for cases where the processes in a group depend on each other and the whole set will fail to function properly if one is killed.

This patch set is in the -mm tree (and thus in linux-next) now, so it is on the path for merging during the next merge window. It has proved to be a relatively controversial feature, though. There are no real objections to teaching the OOM killer about control groups, but there is significant disagreement over just how the OOM killer should treat those groups. Most of these complaints can be found summarized in this message from David Rientjes.

The first of these is that processes in the root control group are treated differently from those in any other group. The memory-size computation is different and, importantly, the `oom_score_adj` value is not used for processes

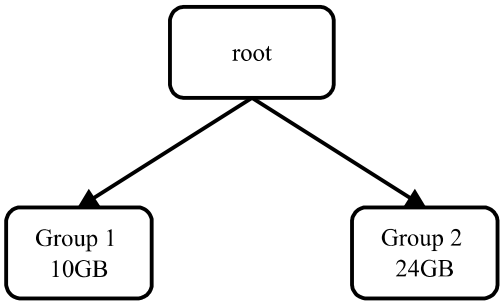
running inside of (non-root) control groups. That can lead to surprising results when it come time for the OOM killer to choose a victim. Rientjes says that the solution is to use the same heuristic for all processes and groups.

Perhaps surprisingly, a number of memory-management developers seem to disagree with this position. In a system dedicated to container workloads, they say, there should be no significant processes running in the root control group; there should be little in the root beyond kernel threads and maybe some system-level daemons. The `oom_score_adj` knob can still be used to ensure that the OOM killer will leave those processes alone. As Johannes Weiner put it:

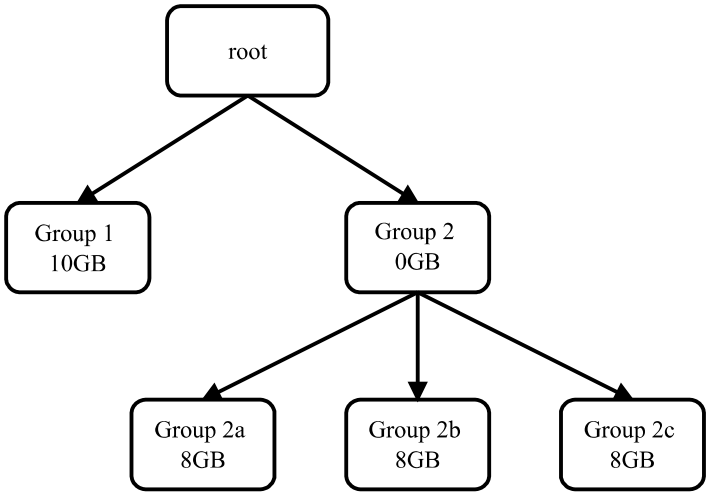
> You don't have any control and no accounting of the stuff situated inside the root cgroup, so it doesn't make sense to leave anything in there while also using sophisticated containerization mechanisms like this group oom setting.

Rientjes finds this argument unconvincing, however.

Another issue Rientjes pointed out is that the new OOM killer is not hierarchical; each control group is considered as a separate entity. Imagine the following simple hierarchy, with the memory usage of each group shown:



If the OOM killer is brought forth, it will quickly conclude that Group 2 is the problem and will target a process found there. Thus far, things are as one might expect. But if the container running in Group 2 creates some subgroups of its own and splits its workload between them, the result could look something like this:



Now, Group 1 will look like the biggest group in the system, and Group 2 will escape the OOM killer's attention. A truly hierarchical view of the control-group hierarchy (which is generally how things are supposed to work) would see the 24GB of memory used by Group 2 and kill a process there instead.

Once again, there is disagreement over whether there is really a problem here or not. Many users of control groups may not want the fully hierarchical behavior. If one were to substitute "Group 1" and "Group 2" with

"Accounting" and "Scientists", for example, it might well seem right that the latter group would use more memory overall. Besides, accountants are always fair game, so the current system behaves as it should.

With regard to the deliberate dodging of the OOM killer by creating subgroups, the response is that such gaming of the system is possible now. Small processes will be passed over, while large processes are targeted, so a clever user could split a task into a large number of processes and get away with using more memory. The control-group-aware mechanism doesn't enable anything new in that regard.

Finally, Rientjes also complained that, since the `oom_score_adj` value is ignored within control groups, there is no longer any way for users to influence how the OOM-killing decision is made. The answer here seems to be that the `oom_score_adj` mechanism is unwieldy and not particularly useful anyway. As Michal Hocko put it:

> oom_score_adj is basically unusable for any fine tuning on the process level for most setups except for very specialized ones. The only reasonable usage I've seen so far was to disable OOM killer for a process or make it a prime candidate. Using the same limited concept for cgroups sounds like repeating the same error to me.

Rientjes, naturally, disagreed, saying: "The ability to protect important cgroups and bias against non-important cgroups is vital to any selection implementation". He further argued that this feature should be incorporated before the new OOM killer goes upstream to avoid changing user-visible behavior in future kernel releases.

Next steps

These concerns notwithstanding, the control-group-aware OOM-killer patches have landed in the -mm tree. That is not an absolute guarantee that they will go into the mainline; -mm maintainer Andrew Morton often puts interesting work there to see what problems turn up. Rientjes has not given up, though; he has been working on a patch series of his own adding the features he would like to see in the new OOM killer. The changes he makes include:

- A new `memory.oom_policy` knob is added to control groups. Setting it to `none` causes the current largest-process heuristic to be used. A setting of `cgroup` will cause the OOM killer to pick the single group with the largest memory usage and kill a process within it; setting the root group's policy to `cgroup` reproduces the behavior of Gushchin's patch set. Finally, a setting of "`tree`" enables a fully hierarchical mode. With this knob, the hierarchical mode is available for those who want it; it is also possible to use different modes for different subtrees of the control-group hierarchy.
- The same heuristic is used to compare processes across all groups, including the root group. When control groups are in use for OOM-killer control, the `oom_score_adj` value is ignored with one exception: setting it to -999 (still) makes the associated process unkillable.

This patch set is not yet in -mm, but there does not appear to be any real opposition to it at this point. It preserves the behavior of the original control-group-aware OOM killer for those who want it while making other modes available "for general use" of the feature. So chances are good that it will be included when the new OOM killer finds its way into the mainline. Of course, chances are equally good that many users will still be unhappy with how the OOM killer works and will be looking for yet another set of heuristics to use — it's a traditional part of Linux kernel development, after all.

Index entries for this article
Kernel Memory management/Out-of-memory handling

(Log in to post comments)

Teaching the OOM killer about control groups

Posted Jul 28, 2018 13:51 UTC (Sat) by **abk77** (guest, #121336) [Link]

Very nice explanation. thank you!. Diagrams/pictures are very useful.

Reply to this comment

OOM killer and cgroups. Better do it in user-space?

Posted Jul 28, 2018 19:09 UTC (Sat) by **darwish** (guest, #102479) [Link]

Thanks a lot Jon for the great article :-)

Part of me feels that this is _too much policy_ leaking into the kernel code. I guess the Facebook approach of relegating this to user-space makes more sense?

▷▷ https://code.fb.com/production-engineering/open-sourcing-...

Maybe something similar can be done in systemd (systemd-oomd?), with its standardized user-space configuration options instead of all of these policy decisions and tunables ..

Reply to this comment

OOM killer and cgroups. Better do it in user-space?

Posted Jul 28, 2018 21:16 UTC (Sat) by **simcop2387** (subscriber, #101710) [Link]

I think making it aware of the groups for this kind of scheduling isn't too much policy. Since the cgroups can affect everything else the kernel does with them (scheduling, io, etc.) it makes sense to extend it to this. And adding the "kill the whole group" bit makes administration easier, have your container all in one cgroup and just let the oom killer take the whole thing out and then your infrastructure restarts it. No need for a complicated heartbeat or other more likely to fail signal saying this container isn't functioning properly anymore. That said, anything beyond making it aware of these structures I think does make sense to put into something userspace. Not sure how facebook's setup works but I could see something that's dynamically adjusting the oom scores to try to keep certain things from being killed while encouraging others and that's going to be a much more complicated policy since it'll be system/service/etc. specific.

Reply to this comment

OOM killer and cgroups. Better do it in user-space?

Posted Jul 28, 2018 23:40 UTC (Sat) by **darwish** (guest, #102479) [Link]

Yeah, I really like the "kill the whole cgroup" option too (memory.oom_group).

It makes sense especially since systemd puts each "service" in its its own cgroup, and killing the whole service sounds much more graceful than picking a single victim process from within it.

Reply to this comment

OOM killer and cgroups. Better do it in user-space?

Posted Jul 31, 2018 21:04 UTC (Tue) by **nilsmeyer** (subscriber, #122604) [Link]

Depends on the service really, what I often see is that there is a main process that fork()s into multiple children and some of the child processes grow out of control. Although this may be handled better within the cgroup?

Reply to this comment

OOM killer and cgroups. Better do it in user-space?

Posted Jul 29, 2018 15:15 UTC (Sun) by **foom** (subscriber, #14868) [Link]

I don't think killing all the processes in the cgroup makes anything appreciably easier. Since any of the processes could crash in some other way, you still need whatever monitoring mechanisms you had before.

Reply to this comment

Killing containers.

Posted Jul 29, 2018 16:09 UTC (Sun) by **rweikusat2** (subscriber, #117920) [Link]

Obviously.

But if a control group happens to represent a container, the processes belonging to it will usually be cooperating more closely than unrelated, other processes also running on the same system. Hence, killing one process running inside a container will end up as more or less drastically malfunctioning virtual server while killing all of them would be more like a clean shutdown. I certainly prefer the latter of the former.

There's also another issue: Assuming the OOM situation was caused by some software bug/ malfunction, chances are that it will happen again fairly quickly as the "monitoring infrastructure" will seek to reestablish the problematic state. Killing everything in the cgroup should prevent that from happening.

Reply to this comment

Teaching the OOM killer about control groups

Posted Jul 29, 2018 21:27 UTC (Sun) by **meyert** (subscriber, #32097) [Link]

How does this related to cgroups with the memory controller? Isn't oom killer also invoked when the cgroups memory limit is hit? Can then"kill the whole cgroup" not also be implemented in the cgroup itself by installing an cgroup oom notification listener?

Reply to this comment

Teaching the OOM killer about control groups

Posted Jul 30, 2018 5:50 UTC (Mon) by **dbe** (guest, #100351) [Link]

Would that mean that a cgroup could ignore an oom notification?

Reply to this comment

Teaching the OOM killer about control groups

Posted Jul 30, 2018 7:42 UTC (Mon) by **vbabka** (subscriber, #91706) [Link]

AFAIU this is not about cgroups memory limit, but a system-wide OOM in a situation where there are cgroups. There might be per-cgroup limits, but it's not that the sum of these limits would equal the system memory, because that would lead to underutilized system. So instead there's overcommit and the workloads can tolerate being killed on the level of a whole group.

Reply to this comment

Teaching the OOM killer about control groups

Posted Aug 1, 2018 13:21 UTC (Wed) by **mstsxfx** (subscriber, #41804) [Link]

> This patch set is not yet in -mm, but there does not appear to be any real opposition to it at this point.

Well, this is not entirely true. There were quite serious concerns about the proposed API allowing for weird corner cases. E.g. http://lkml.kernel.org/r/20180130085013.GP21609@dhcp22.su.... The discussion circled in some repetitive arguments so it is not really easy to follow up, unfortunately. The last version hasn't been reviewed AFAIK but bases are quite similar so I am skeptical this is mergeable anytime soon.

The biggest quarrel about this whole thing is a different view about feature completeness IMHO.

While the original Roman's proposal was targeting very specific class of usecases (containers as you've mentioned) and it was an opt-in so those uninterested could live with the original policy/heuristic with potential extensions[1] to be done on top.

David was pushing hard for a more generic solution which would give a bigger power over the oom selection policy to userspace. While this is a good thing in general, the primary problem is that this is extremely hard to get right. We have been discussing this for years without finding moving forward much because opinions of what is really important vary a lot.

[1] group_oom makes a lot of sense regardless of the oom victim selection policy because some workloads are inherently indivisible

Michal Hocko

Reply to this comment

Copyright © 2018, Eklektix, Inc.

This article may be redistributed under the terms of the Creative Commons CC BY-SA 4.0 license

Comments and public postings are copyrighted by their creators.

Linux is a registered trademark of Linus Torvalds